

# Alice in Parallel

## Fast Recurrence

Gary Nan Tie

Jul 18, 2026

### Abstract

A fast parallel version of Alice, a state machine LLM that combines a recurrent implicit deep learning model transition with an attention-based output over learned fixed points, optimized for syntactic prediction and semantic coherence within a word embedding space.

### Preliminaries

A **word embedding** is typically a representation of a word in a vector space. The representation encodes the meaning of words in such a way, that words closer in the vector space are expected to be similar in meaning. For example, the word embedding `word2vec`, captures information about the meaning of a word based on the surrounding words, syntactic co-occurrence.

The Chebyshev metric, for  $n$ -vectors  $a$  and  $b$ , is defined by  $\text{Cheb}(a,b) := \max_{i \in [n]} (|a_i - b_i|)$ .

Let  $\text{Emb}: \mathbb{R}^p \rightarrow \mathbb{R}^{p'}$  be a chosen word embedding, mapping token  $t_i$  to vector  $v_i$ ,  $i \in [m]$ . Partition  $[p']$  into  $B$  blocks, so  $v_i = [1v_i, 2v_i, \dots, Bv_i]'$ .

### Synopsis

Think of a state machine unrolled as a state backbone with input-output ribs. We will pre-train a backbone to simulate a word embedding, using parallel recurrent implicit deep equilibrium models. Parallel attention layer ribs perform LLM inference on this backbone that carries corpus contextual meaning. Simply put, transformer attention grafted onto coherent semantic roots.

### Backbone

Given input token string  $t_1, \dots, t_m$  where  $t_i \in \mathbb{R}^p$ ,

we will construct a parallel state machine **Ecila(S, O')**  
 with transition function  $S(h_{i-1}, t_i)$  and output function  $O'(h_i)$ .  
 Designed to simulate Emb,  $Ecila(S, O')$  will be defined  
 by  $B$  parallel state machines  $jEcila(jS, jO)$ ,  
 with  $S := 1S \mid \dots \mid jS \mid \dots \mid BS$  and  $O' := 1O \mid \dots \mid jO \mid \dots \mid BO$   
 (See Appendix).

**Notation:** Let  $q = Bb$ . For  $i \in [m]$ , let  
 $h_i = [1h_i, 2h_i, \dots, Bh_i]' \in R^q$ , with  $j h_i \in R^b$ ,  $j \in [B]$ , and  
 $k_i = [1k_i, 2k_i, \dots, Bk_i]' \in R^q$ , with  $j k_i \in R^b$ ,  $j \in [B]$ .  
 With transition  $h_i = S(h_{i-1}, t_i)$  and  
 output  $O'(h_i) = o'_i = [1o'_i, 2o'_i, \dots, Bo'_i]'$ ,  $Ecila(S, O')$   
 will approximate  $Emb(t_i) = v_i = [1v_i, 2v_i, \dots, Bv_i]'$ ,  
 in parallel blocks.

Define initial hidden state  $h_0 = [\epsilon_1, \dots, \epsilon_B]' \in R^q$ ,  
 where independent draws  $\epsilon_j$  are from  $Normal(0,1)$ .

### Parallel implicitRNNs $jS$ to define $S$

Let  $j \in [B]$ ,  
 $j h_0 := \epsilon_j$ ,  $j k_0 := 0$

Let  $i \in [m]$ ,  
 $j u_i := [t_i \ j h_{i-1}]' \in R^{(p+b)}$

$j k_i := j \phi (j A \ j k_i + j B \ j u_i)$  (equilibrium equation)  
 $j h_i := j C \ j k_i + j D \ j u_i$  (prediction equation)

where  $j A := j \Gamma - \text{diag} (|j \Gamma| \ \mathbf{1}) + \gamma \ \mathbf{I}_d$ ,  $0 < \gamma < 1$ ,  
 ensures  $j A$  well-posed for  $j \phi$ , that is, the fixed  
 point equation has a solution and it is unique.

Let  $j \theta' = \{ (W_1, b_1), (W_2, b_2) \}$  denote parameters  
 for a **feedforward network** defined by  
 $FFNj \theta' (j h_i) := \text{ReLU} (j h_i W_1 + b_1) W_2 + b_2$

Define  $j$ -th transition  $j S(j h_{i-1}, t_i) := j h_i$  and  
 $j$ -th output  $j o_i = j O(j h_i) := FFNj \theta' (j h_i)$ , to

approximate  $jv_i$ .

### **jEcila(jS, jO) training problem**

For each  $j \in [B]$ , in parallel learn implicitRNN matrices  $jA, jB, jC, jD$  and fixed points  $jki$  for  $jhi$ , and feedforward network parameters  $j\theta'$  to minimize:  
 $\sum_i \in [m] \text{Cheb}(joi, jvi)$

Thus to simulate  $\text{Emb}(t_i) = v_i = [1v_i, 2v_i, \dots, Bv_i]'$ , we learn  $\text{Ecila}(S, O')$  trained by  $B$  state machines  $j\text{Ecila}(jS, jO)$  in parallel.

### **Ribs**

**Alice** is another state machine with transition and output functions  $(S, O)$ , where  $S$  is defined by  $\text{Ecila}$  the state machine above, and state  $h_i$  viewed as a contextual representation of token  $t_i$ , is now held fixed.

Given a token sequence  $t_1, t_2, \dots, t_i$  the goal is to find a token continuation  $o_{i+1}$  with certain desired properties (perplexity and Renyi  $\alpha$ -entropy). To achieve this, choose the  $o_{i+1}$  to be Alice output  $O(h_i)$ , yet to be defined, given by attention layers designed to have these desired properties.

### **Alice attention layer(s)**

Input  $h_i$  from  $\text{Ecila}(S)$ .

Let score be a desired similarity function, like a reproducing kernel.

For  $i \in [m]$ , define probabilities

$\alpha_{ij} := \text{soft max over } j \in [i] \text{ score}(h_i, k_j)$ .

Let  $h_i^\wedge = \sum_j \in [i], \alpha_{ij} v_j$  and define update  $h_i' := \text{FFN}\theta(h_i^\wedge)$ .

Output  $h_i'$ .

Stack  $L$  attention layers  $\Theta = (\theta_1, \dots, \theta_L)$  composing updates and now let  $h_i'$  denote the final  $L$ -th update.

For  $i \in [m]$ , define probabilities

$\beta_{ij} := \text{soft max over } j \in [i] \text{ score}(h_i', k_j)$

and let  $j^*i := \arg \max \text{ over } j \in [i], \beta_{ij}$ .

Define token continuation  $o_{i+1} = O(h_i) := t_{j^*i}$ ,  
the token whose fixed point  $k$  to date is most similar  
(probable under distribution  $\beta$ ) to update  $h'$ .

Define the average Renyi  $\alpha$ -entropy of  $\{t_1, \dots, t_m\}$ ,  
a measure of semantic alignment by:  
 $E(\{t_i\}) := (1/m) \sum_i H(i, \alpha)$ ,  
where  $H(i, \alpha) := H_\alpha(\beta_{ij})$ , the Renyi  $\alpha$ -entropy of  $\{\beta_{ij}\}$ .

Together perplexity of output  $O$  and average Renyi  $\alpha$ -entropy  $E$  train the  $h_i$  to holistically capture  
syntax and semantics.

### **Alice training problem cf [1]**

Let  $\lambda > 0$ , learn feedforward network parameters  $\Theta$  to minimize:

$$[ \text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\}) ]$$

Remarks:

$O$  next-token continuation  $t_1, \dots, t_i$ ,  $o_{i+1}$  reflects the  
word embedding through the  $h_i^*$ .

Perplexity intrinsically evaluates the continuation,  
regularized by average Renyi  $\alpha$ -entropy.

We have estimated  $Alice(S, O)$  in stages, first learning  $S$  from training  $Ecila(S, O')$ , then holding  
the  $\{h_i\}$  from  $S$  fixed, discarded  $O'$  and learned attention ribs  $O$ . Here,  $Alice$  is learned in stages,  
 $S$  then  $O$ , not  $(S, O)$  jointly.

### **Next-token prediction**

Initialization: Let  $t_1$  = the empty token or a seed token

Given  $t_1$ , define  $t_2 = o_2$ .

Given  $t_1, t_2$ , define  $t_3 = o_3$ .

Given  $t_1, t_2, t_3$ , define  $t_4 = o_4$  and so on ...

autoregressively generating next-token continuation.

### **Sequence-to-Sequence learning**

First, we solve the next-token problem for both the  
target and source tokens thereby learning contextual



token representations for target and source. We can then directly compare their state similarity to determine a best translation based on meaning rather than surface syntax alone.

## **Summary**

Pre-training Ecila(S, O') in parallel to learn a word embedding, enables fast LLM inference. We learn token continuations on the fly from state machine Alice(S, O) attention layers in parallel. For the state machine schematics see the Appendix.

**Alice**, a fast parallel state machine LLM that combines a recurrent implicit deep learning model transition with an attention-based output over learned fixed points, is optimized for syntactic prediction and semantic coherence within a word embedding space. This state machine foundation grounds transformer attention in meaning. Surprisingly scaling works, but ultimately meaning matters more!

## **References**

[1] 'Alice Through the Word Embedding'

DOI: 10.13140/RG.2.2.34901.79841

[https://www.researchgate.net/publication/408460384\\_Alice\\_Through\\_the\\_Word\\_Embedding?channel=doi&linkId=6a4921a95ec65810298eecaf&showFulltext=true](https://www.researchgate.net/publication/408460384_Alice_Through_the_Word_Embedding?channel=doi&linkId=6a4921a95ec65810298eecaf&showFulltext=true)

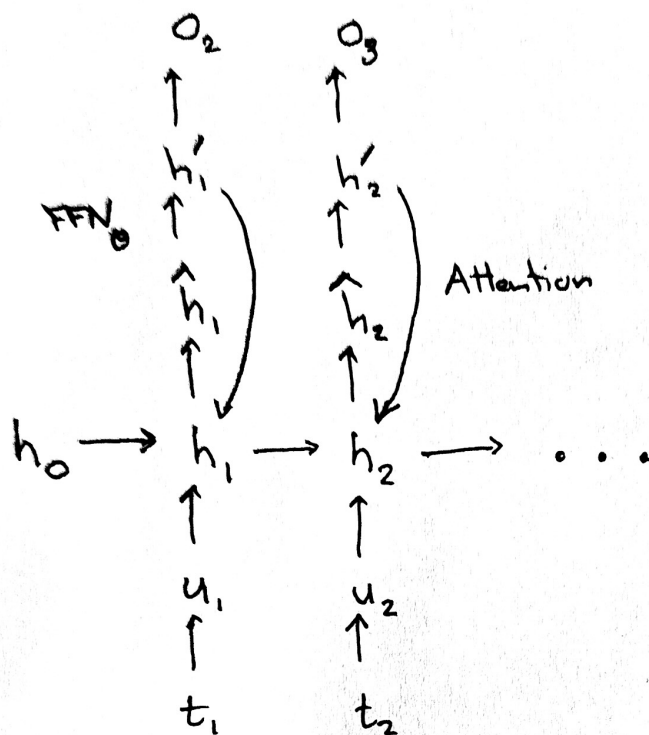
[2] 'Strategies for learning token continuations'

DOI: 10.13140/RG.2.2.16884.31366

[3] 'State deformation from word embedding to LLM'

DOI: 10.13140/RG.2.2.26347.71204

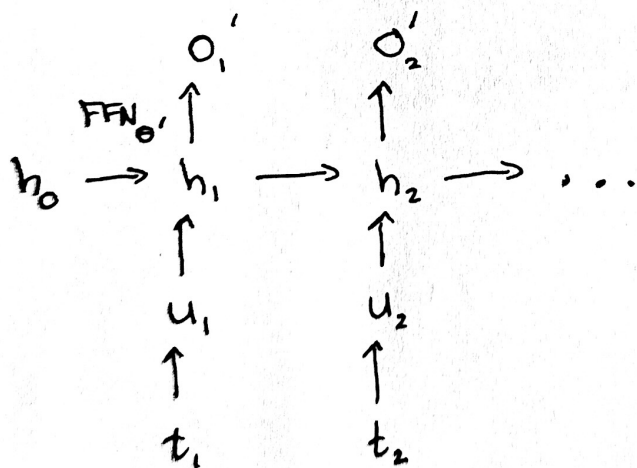
## Appendix - State Machines



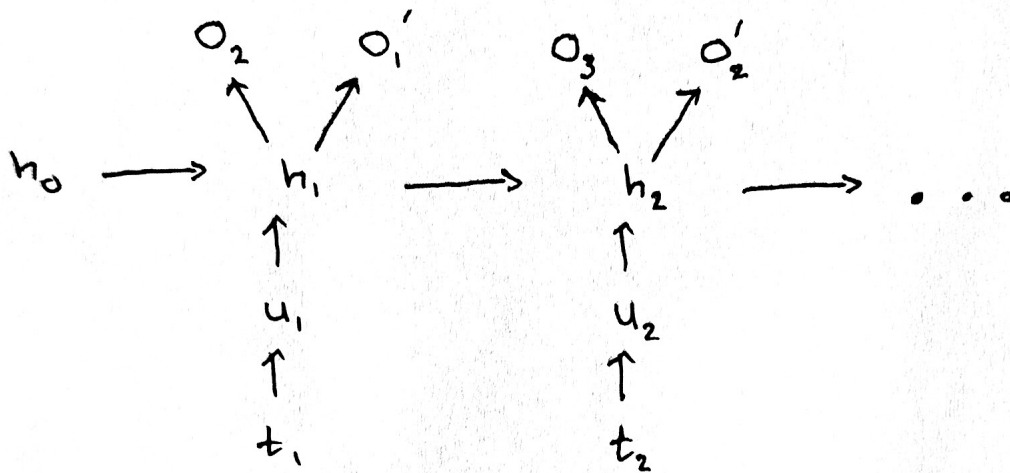
Alice ( $\mathcal{S}, \mathcal{O}$ )

- pre-train  $\{h_i\}$

- learn  $\{o_j\}$  in parallel



Ecila ( $\mathcal{S}, \mathcal{O}'$ )



AliceCila  $(S, \oplus, \oplus')$

—

### State Machine $(\mathcal{S}, \mathcal{O})$

Input: target tokens  $t_1, \dots, t_m \in \mathbb{R}^p$

Initial state:  $h_0 = \mathbf{0} \in \mathbb{R}^q$

Next state:  $h_i = \mathcal{S}(h_{i-1}, t_i) \in \mathbb{R}^q, \quad i \in [m]$

Output:  $o_{i+1} = \mathcal{O}(h_i) \in \mathbb{R}^p, \quad i \in [m]$

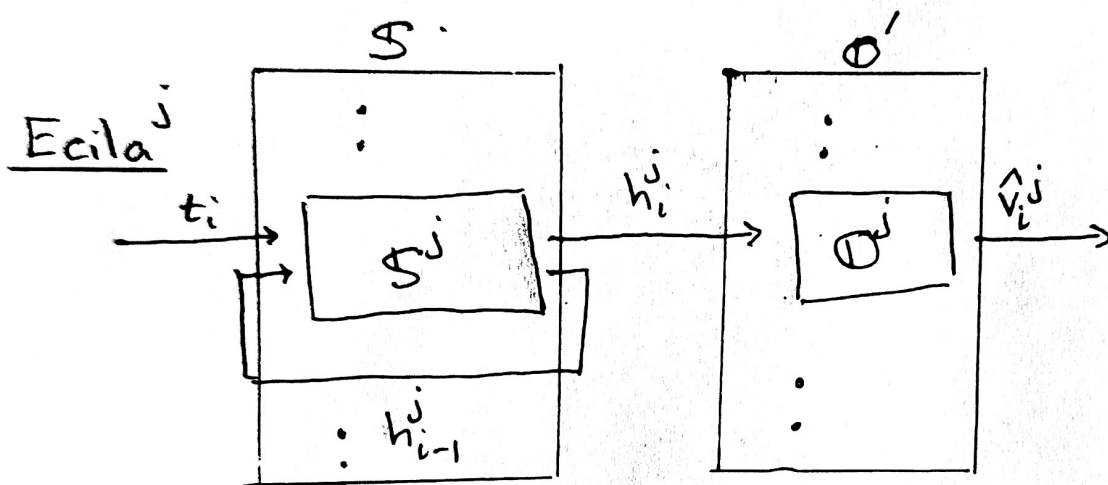
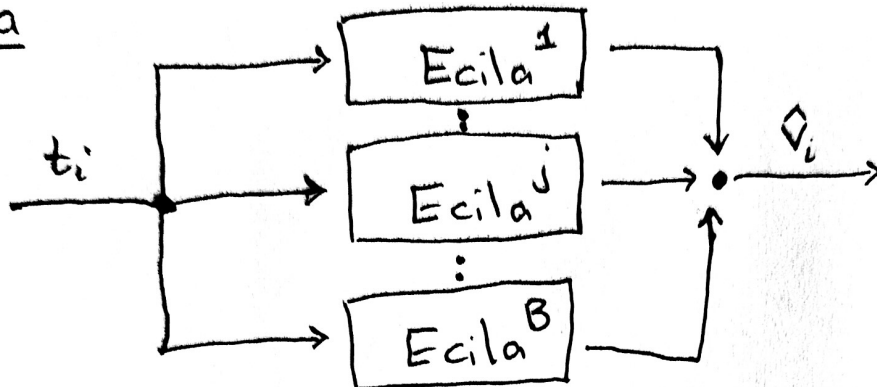
where  $\mathcal{S}$  is a transition function and

$\mathcal{O}$  is an output function.

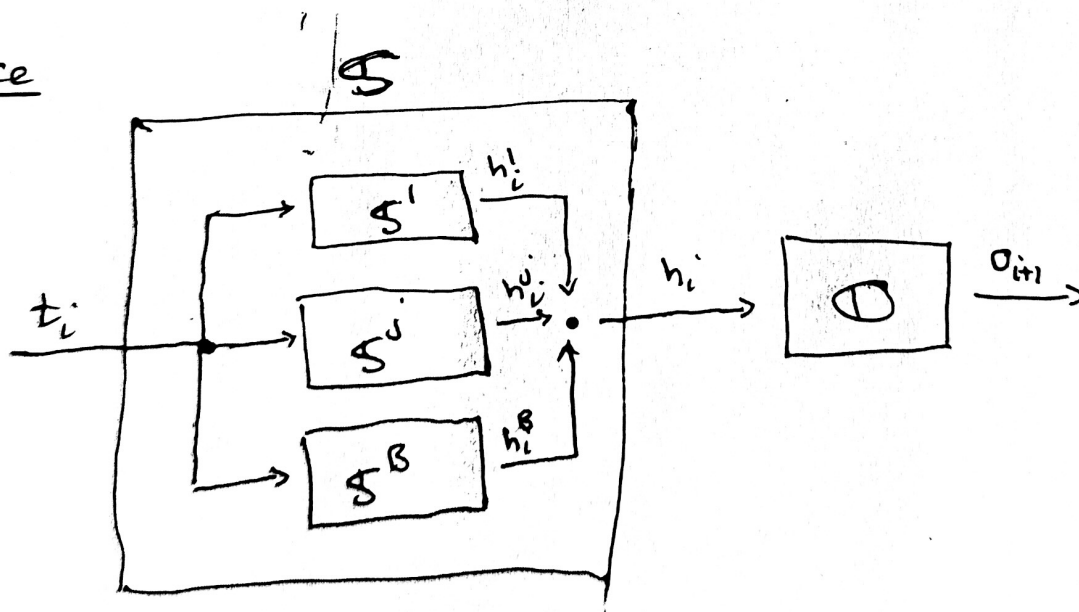
—

# Appendix - Parallelization

Ecila



Alice



In parallel,  $S: S^1 | \dots | S^B$      $\Theta': \Theta^1 | \dots | \Theta^B$

Let  $\text{Emb} : \mathbb{R}^P \rightarrow \mathbb{R}^{P'}$  be a word embedding

$$t_i \mapsto v_i = \begin{bmatrix} v_i^1 \\ \vdots \\ v_i^B \end{bmatrix}$$

where  $v_i$  has been partitioned into  $B$  blocks.

Given input token string  $t_1, \dots, t_m \in \mathbb{R}^P$

we will define a state machine  $\text{Ecila}^j(\mathcal{S}^j, \mathcal{O}^j)$

with transition function  $\mathcal{S}^j(h_{i-1}^j, t_i)$  and

output function  $\mathcal{O}^j(h_i^j)$ , where  $h_i^j$  is chosen

so that  $\hat{v}_i^j \triangleq \mathcal{O}^j(h_i^j) \approx v_i^j$ .

Define initial hidden state  $h_0 = \begin{bmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_B \end{bmatrix} \in \mathbb{R}^q$ ,  $q = Bb$ ,

where independent draws  $\varepsilon_j \sim \text{Normal}_{\mathbb{R}^b}(0, 1)$

For  $j \in [B]$ , implicitRNN  $\mathcal{S}^j$ :

$$h_0^j \triangleq \varepsilon_j \in \mathbb{R}^b, \quad k_0^j = 0 \in \mathbb{R}^b$$

For  $i \in [m]$ ,  $u_i^j = \begin{bmatrix} t_i \\ h_{i-1}^j \end{bmatrix} \in \mathbb{R}^{p+b}$ , let  $r = p+b$

$$k_i^j \triangleq \varphi^j \left( \underset{b \times b}{A^j} k_i^j + \underset{b \times r}{B^j} u_i^j \right) \in \mathbb{R}^b$$

$$h_i^j \triangleq \underset{b \times b}{C^j} k_i^j + \underset{b \times r}{D^j} u_i^j \in \mathbb{R}^b$$

where  $A^j = \Gamma^j - \text{diag}(|\Gamma^j| \mathbb{1}_b) + \gamma \text{Id}_b$ ,  $0 < \gamma < 1$ ,

ensures  $A^j$  is well-posed for  $\varphi^j$

Define transition function  $\mathcal{S}^j(h_{i-1}^j, t_i) \triangleq h_i^j$

Let  $\Theta_j' = \{(W_1, b_1), (W_2, b_2)\}$  denote parameters

for FeedForward network:

$$\text{FFN}_{\Theta_j'}(h_i^j) \triangleq \text{ReLU}(h_i^j W_1 + b_1) W_2 + b_2$$

Define output function  $\hat{v}_i^j = o_i^j = \mathcal{O}^j(h_i^j) = \text{FFN}_{\Theta_j'}(h_i^j)$

This completes the definition of state machine  $\text{Ecila}^j(\mathcal{S}^j, \mathcal{O}^j)$ .

Recall the Chebyshev metric for  $n$ -vectors:

$$\text{Cheb}(a, b) = \max_{i \in [n]} (|a_i - b_i|)$$

Ecila<sup>j</sup> ( $\Phi^j, \mathcal{O}^j$ ) training problem

Learn implicit RNN matrices  $A^j, B^j, C^j, D^j$ ,  
and fixed points  $k_i^j$ , and feedforward network  
parameters  $\Theta_j'$  to minimize:

$$\sum_{i=1}^m \text{Cheb}(\sigma_i^j, v_i^j)$$

where  $\sigma_i^j = \mathcal{O}^j(h_i^j) = \text{FFN}_{\Theta_j'}(h_i^j)$

and  $h_i^j = C^j k_i^j + D^j u_i^j$ .

$$k_i^j = \varphi^j(A^j k_i^j + B^j u_i^j)$$



$Ecila(\mathcal{S}, \mathcal{O}')$  is the parallel state machine:

$$\text{Parallel}(Ecila^1, \dots, Ecila^B)$$

$$\text{with } \mathcal{S} = \mathcal{S}' | \dots | \mathcal{S}^B$$

$$\text{and } \mathcal{O}' = \mathcal{O}' | \dots | \mathcal{O}^B$$

$Ecila(\mathcal{S}, \mathcal{O}')$  is learned by  $B$  state machines

$(Ecila^j(\mathcal{S}^j, \mathcal{O}^j))$  in parallel to simulate Emb.

From  $Ecila(\mathcal{S}, \mathcal{O}')$  keep  $\{h_i\}_{i=1}^m$  where  $h_i = \begin{bmatrix} h_i^1 \\ \vdots \\ h_i^B \end{bmatrix} \in \mathbb{R}^q$

that is discard  $\mathcal{O}'$ , and define  $Alice(\mathcal{S}, \mathcal{O})$  [1]

as before by  $o_{it+1} = \mathcal{O}(h_i) \triangleq t_{j_i^B}$  from attention layers.

The Alice training problem given  $\{h_i\}$ : Let  $\lambda > 0$ ,

learn FeedForward network parameters  $\Theta$  to minimize:

$$\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\})$$

So here,  $Alice(\mathcal{S}, \mathcal{O})$  is learned in stages, not jointly.